

Weather Derivatives Project

Samuele Caneschi — MSc in Finance & Energy Markets — UniBo

This project demonstrates the theoretical foundations learned during Prof. Romagnoli's Financial Mathematics course and Prof. Cordioli's Python Coding & Data Science course (AY 2025/2026).

We model daily temperatures via a CARMA process, change measure via Girsanov's theorem, and price a HDD call option under Q.

ARMA model

To get a python function that simulates temperature as a continuous-time stochastic process, we implement equations from CARMA process.

- 1st step is to model the trend:

$$\Lambda(t) = a_0 + a_1 t + a_2 \cos\left(\frac{2\pi t}{365}\right) + a_3 \sin\left(\frac{2\pi t}{365}\right)$$

```
In [9]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42) # reproducibility

def trend(t, a0, a1, a2, a3):
    #where a0 is the mean temperature, a1 is the linear trend (climate ch
    return a0 + a1*t + a2*np.cos((2*np.pi*t)/365) + a3*np.sin((2*np.pi*t)
```

```
In [10]: # Solve the function with Florence's data
t = np.arange(0, 730, 1) #two years
a0 = 15.5
a1 = 0.00012 # 'a' are referred to Florence, Italy
a2 = -8.6
a3 = -3.8
lambda_Florence = trend(t, a0, a1, a2, a3)
```

- 2nd step is to implement the stochastic engine, a CAR(1) process (OU), which includes mean reversion. Differential equation is:

$$dX(t) = -\alpha X(t)dt + \sigma dB(t)$$

```
In [11]: def car1(n, alpha, sigma): #n = number of days, alpha = return to mean s
dt = 1
X = np.zeros(n)
for i in range(1, n):
    dX = -alpha * X[i - 1] * dt + sigma * np.random.normal()
    X[i] = X[i - 1] + dX
return X
```

```
In [12]: # Solve the function with Florence's data
n = len(lambda_Florence)
epsilon_florence = carl(n, alpha=0.1, sigma=2)
```

- 3rd step consists in assembling a df with trend and stochastic engine:

```
In [13]: final_temperatures = lambda_Florence + epsilon_florence
df_quant = pd.DataFrame({'Day': t, 'Trend': lambda_Florence, 'Final Tempe
df_quant['HDD'] = (18 - df_quant['Final Temperature']).clip(lower=0)
df_quant['CDD'] = (df_quant['Final Temperature'] - 18).clip(lower=0)
display(df_quant.head())
```

	Day	Trend	Final Temperature	HDD	CDD
0	0	6.900000	6.900000	11.100000	0.0
1	1	6.835983	7.829412	10.170588	0.0
2	2	6.774534	7.392091	10.607909	0.0
3	3	6.715671	8.566849	9.433151	0.0
4	4	6.659410	11.371530	6.628470	0.0

CARMA model

Let's build a CARMA(p,q) process, composed of a drift matrix A, entrance vector (ep) and an observation vector (b):

$$dX(t) = A \cdot X(t)dt + e_p \cdot \sigma(t)dB(t)$$

```
In [14]: class CARMA_process:
    def __init__(self, ar_coeffs, ma_coeffs, sigma):
        self.p = len(ar_coeffs)
        self.q = len(ma_coeffs) - 1
        self.sigma = sigma

        if self.p <= self.q:
            raise ValueError('Q must be smaller than P')

        # Matrix A
        self.A = np.zeros((self.p, self.p))
        if self.p > 1:
            self.A[:-1, 1:] = np.eye(self.p - 1)
            self.A[-1, :] = -np.array(ar_coeffs)[::-1]

        # Vector e_p
        self.e_p = np.zeros(self.p)
        self.e_p[-1] = 1

        # Vector b
        self.b = np.zeros(self.p)
        self.b[self.q + 1:] = np.array(ma_coeffs)

    def simulate(self, n_days, dt=1):
        # State matrix X(t)
        X = np.zeros((self.p, n_days))
```

```

dw = np.random.normal(0, np.sqrt(dt), n_days)

for t in range(1, n_days):
    drift = np.dot(self.A, X[:, t - 1]) * dt
    shock = self.e_p * self.sigma * dw[t]
    X[:, t] = X[:, t - 1] + drift + shock

Y = np.dot(self.b, X)
return Y

```

In [15]: *# Plug in your values*

```

ar_coeffs = [0.5, 0.04]
ma_coeffs = [1, 0.2]
sigma = 2.5
n_days = 365

```

In [16]: *# Execute CARMA*

```

weather_model = CARMA_process(ar_coeffs, ma_coeffs, sigma)

```

In [17]: *# Find fluctuations*

```

fluctuations_Y = weather_model.simulate(n_days)

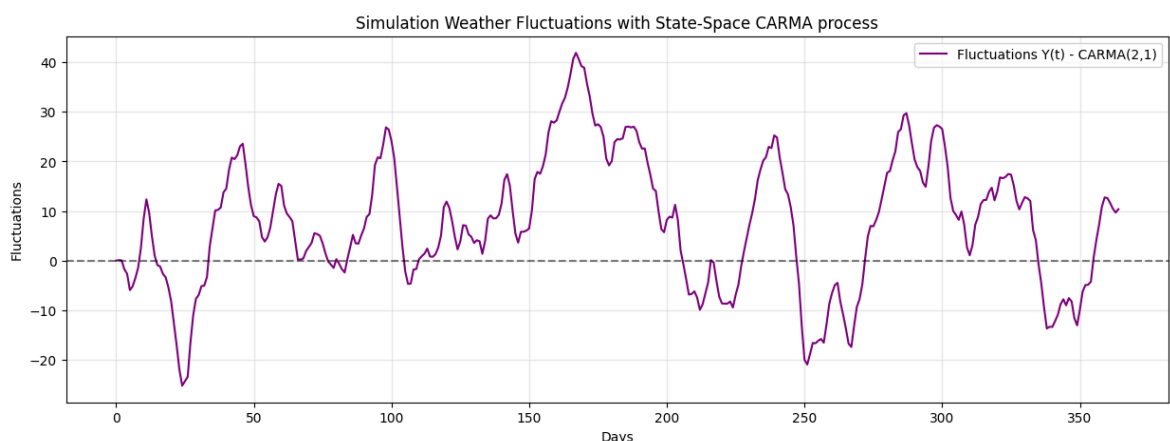
```

In [18]: *# Data visualization*

```

plt.figure(figsize=(15, 5))
plt.plot(fluctuations_Y, label="Fluctuations Y(t) - CARMA(2,1)", color="p")
plt.axhline(0, color='black', linestyle='--', alpha=0.5)
plt.title("Simulation Weather Fluctuations with State-Space CARMA process")
plt.xlabel("Days")
plt.ylabel("Fluctuations")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```



Now let's change measure in order to get a risk-neutral probability and being able to price derivatives. Girsanov's theorem will come to the rescue:

$$A^Q = A + e_p \cdot \theta'$$

In [19]: **class** CARMA_Pricer(CARMA_process):

```

    def simulate_Q_paths(self, n_days, theta_vector, n_sims=10000, dt=1.0)
        if len(theta_vector) != self.p:

```

```

        raise ValueError(f'Vector theta must be equal to p')
    A_Q = self.A + np.outer(self.e_p, theta_vector)

    X = np.zeros((self.p, n_sims, n_days))

    dW = np.random.normal(0, np.sqrt(dt), (n_sims, n_days))

    for t in range(1, n_days):
        drift = np.tensordot(A_Q, X[:, :, t - 1], axes=([1], [0])) *
        shock = np.outer(self.e_p, dW[:, t]) * self.sigma
        X[:, :, t] = X[:, :, t - 1] + drift + shock
    #  $Y(t) = b' * X(t)$ 
    Y = np.tensordot(self.b, X, axes=([0], [0]))

    return Y

```

```

In [20]: ar_params = [0.5, 0.04]
        ma_params = [1.0, 0.2]
        sigma = 2.5
        n_days = 150

        theta_vector = np.array([0.02, 0.01])

        pricer = CARMA_Pricer(ar_params, ma_params, sigma)

        fluctuations_Q = pricer.simulate_Q_paths(n_days, theta_vector, n_sims=100

```

```

In [21]: # Reconstruct the deterministic trend (e.g., Florence calibrated parameter
        t_winter = np.arange(0, n_days, 1)
        # Calculate real temperatures (Trend + Q-measure fluctuations)
        # IMPORTANT: slice lambda_Florence to match contract length (n_days=150,
        simulated_temperatures = lambda_Florence[:n_days] + fluctuations_Q

        # Calculate daily HDDs (Threshold: 18°C)
        daily_hdd = np.clip(18 - simulated_temperatures, a_min=0, a_max=None)

        # Sum along the horizontal axis to get total HDD per winter scenario
        total_hdd_per_winter = np.sum(daily_hdd, axis=1)

        # Calculate the Fair Forward Price (Mean) and the 95th Percentile (Value
        forward_price = np.mean(total_hdd_per_winter)
        worst_case_scenario = np.percentile(total_hdd_per_winter, 95)

```

VISUALIZATION DASHBOARD

```

In [22]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

        # --- PLOT 1: SPAGHETTI PLOT (Temperature Paths) ---
        # Plot only the first 100 scenarios to save memory
        for i in range(100):
            ax1.plot(t_winter, simulated_temperatures[i, :], color='royalblue', a

        # Add the mean trajectory of all 10,000 simulations (Red Line)
        ax1.plot(t_winter, np.mean(simulated_temperatures, axis=0), color='red',
        ax1.axhline(18, color='black', linestyle='--', linewidth=1.5, label='HDD

        ax1.set_title('Simulated Temperature Paths (Sub-sample: 100)')
        ax1.set_xlabel('Days')

```

```

ax1.set_ylabel('Degrees (°C)')
ax1.legend()
ax1.grid(alpha=0.3)

# --- PLOT 2: HISTOGRAM (HDD Distribution) ---
# Plot the distribution of all 10,000 results
ax2.hist(total_hdd_per_winter, bins=50, color='darkorange', edgecolor='bl

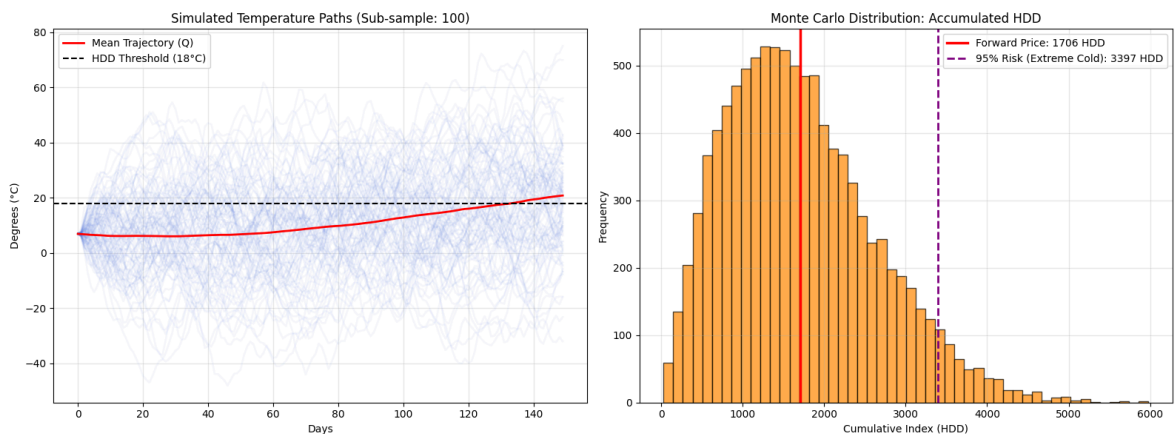
# Forward Price Line (Expected Value)
ax2.axvline(forward_price, color='red', linestyle='-', linewidth=2.5,
            label=f'Forward Price: {forward_price:.0f} HDD')

# Extreme Risk Line (95th Percentile)
ax2.axvline(worst_case_scenario, color='purple', linestyle='--', linewidth=2.5,
            label=f'95% Risk (Extreme Cold): {worst_case_scenario:.0f} HD

ax2.set_title('Monte Carlo Distribution: Accumulated HDD')
ax2.set_xlabel('Cumulative Index (HDD)')
ax2.set_ylabel('Frequency')
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



HDD Call option pricing

$$V^0 = e^{-rT} \cdot \mathbb{E}^{\mathbb{Q}} \left[tick \cdot \max \left(\sum_t HDD_t - K, 0 \right) \right]$$

```

In [23]: TICK = 1_000 # €1,000 per HDD unit (standard CME convention)
K = forward_price # ATM strike = forward HDD
r = 0.0361 # risk-free rate
T = n_days / 365.0 # contract length in years

payoffs = TICK * np.maximum(total_hdd_per_winter - K, 0)
option_price = np.exp(-r * T) * np.mean(payoffs)
se = np.exp(-r * T) * np.std(payoffs, ddof=1) / np.sqrt(len(payoffs))

print(f"HDD Call Option Price : €{option_price:,.2f}")
print(f"95% CI : [€{option_price - 1.96 * se:,.2f}, €{opt
print(f"Strike (ATM) : {K:.1f} HDD")

```

HDD Call Option Price : €360,824.42
95% CI : [€349,197.73, €372,451.10]
Strike (ATM) : 1705.5 HDD